

Outline

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

Outline

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

Outline

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

Solving a Problem

- Problem-solving agent: a type of goal-based agent
 - Decide what to do by finding sequences of actions that lead to desirable states $\implies$ solution
 - **Solution** is a sequence of world state in which the final state satisfy the goal or solution is action sequence in which the last action will result the goal state.
 - The process of looking for such a sequence of actions is called **search**
- Rational agents or Problem-solving agents in AI mostly used search strategies or algorithms to solve a specific problem and provide the best result.
 - **Formalize the problem:** Identify the collection of information that the agent will use to decide what to do.
 - **Define states:** States describe distinguishable stages during the problem-solving process

Solving a Problem

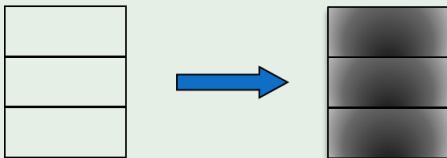
- Define the available **operators/rules** for getting from one state to the next
 - Operators cause an action that brings transitions from one state to another by applying on a current state
- Suggest a **suitable representation** for the problem space/state space: graph, table, list, set or a combination of them

State Space of a Problem

- The **state space** defines the set of all relevant states reachable from the initial state by (any) sequence of actions through iterative application of all permutations and combinations of operators
- A **state space** represents a problem in terms of states and operators that change states.
- **State space** (also called search space/problem space) of the problem includes the various states
 - **Initial state**: defines where the agent starts or begins its task
 - **Goal state**: describes the situation the agent attempts to achieve
 - **Operators /actions** : describe what the agent can execute
 - **Transition states**: other states in between initial and goal states

Coloring Problem

There are 3 rectangles. All are initially white. The problem is to change all the rectangles with white color to black color. Color change is one rectangle at a time. Define state space for coloring problem.



The 8 Puzzle Problem

- there's no other tile blocking you in the direction of the movement; and
- you're not trying to move outside of the boundaries/edges.

1	2	3
8	4	5
7	6	



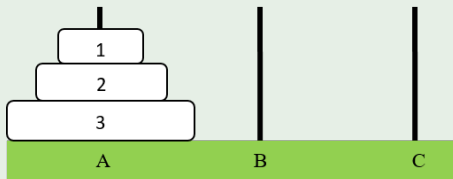
1	2	3
8		4
7	6	5

Examples for Defining State Space of a Problem

Tower of Hanoi

Build the tower on the third peg (on peg C), obeying the following rules:

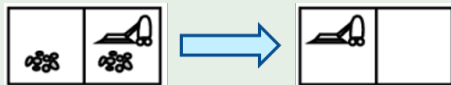
- move one disk a time,
- do not stack bigger disk on smaller one.



Examples for Defining State Space of a Problem

Vacuum World Problem

- The world has only two locations. Each location may or may not contain dirt. The agent may be in one location or the other.
- Three possible actions (Left, Right, Suck). Suck operator clean the dirt, Left and Right operators move the agent from location to location.
- Goal: to clean up all the dirt



Exercises

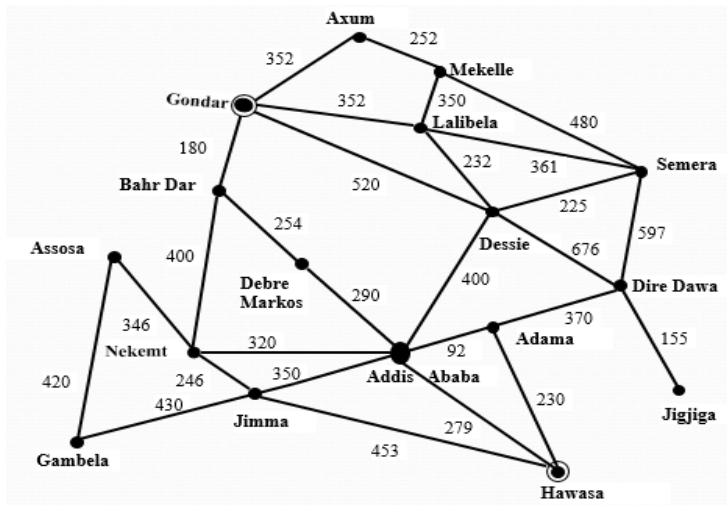
- ① **Missionary-and-Cannibal Problem:** Three missionaries and three cannibals are on one side of a river that they wish to cross. There is a boat that can hold one or two people. Find an action sequence that brings everyone safely to the opposite bank (i.e. Cross the river). But you must never leave a group of missionaries outnumbered by cannibals on the same bank (in any place).
- ② **Water Jug Problem:** We have one 3 liter jug, one 5 liter jug and unlimited supply of water. The goal is to get exactly one liter of water in either of the jug. Either jug can be emptied, filled or poured into the other.

For the above-mentioned problems:

- Identify the set of states and operators
- Show using suitable representation the state space of the

problem

Example: Road Map of Ethiopia



Example: Road Map of Ethiopia

- Current position of the agent: Hawasa.
- Needs to arrive to: Gondar
- Formulate goal:
 - be in Gondar
- Formulate problem:
 - states: various cities actions: drive between cities
- Find solution:
 - sequence of cities, e.g., Hawasa, Adama, Addis Ababa, Dessie, Gondar

Well-defined Problems and Solutions

A well defined problem is a problem in which

- The start state of the problem
- Its goal state
- The possible actions (operators that can be applied to make move from state to state)
- The constraints upon the possible action to avoid invalid moves (this defines legal and illegal moves)

are known in advance

- To define a problem, we need the following elements:
 - states
 - operators
 - goal test function
 - cost function

Well-defined Problems and Solutions

Initial state

- The Initial state is the state that the agent starts in or begins with.
- Example- the initial state for each of the following:
 - Coloring problem: All rectangles white
 - Route finding problem: The point where the agent start its journey (Hawasa)
 - 8-puzzle problem ??

Well-defined Problems and Solutions

Operators

- The set of possible actions available to the agent, i.e.
 - which state(s) will be reached by carrying out the action in a particular state
 - **A Successor function $S(x)$**
 - Is a function that returns the set of states that are reachable from a single state by any single action/operator
 - Given state x , $S(x)$ returns the set of states reachable from x by any single action
- **Example:**
 - Coloring problem: Paint with black color $\rightarrow$ paint (color w , color b)
 - Route finding problem: Drive through cities/places $\rightarrow$ drive (place x , place y)
 - 8-puzzle ??

Well-defined Problems and Solutions

Goal Test Function

- The agent execute to determine if it has reached the goal state or not
 - Is a function which determines if the state is a goal state or not
- **Example:**
 - Route finding problem: Reach Gondar $\rightarrow$ IsGoal(x , Gondar)
 - Coloring problem: All rectangles black $\rightarrow$ IsGoal(rectangle[], n)
 - 8-puzzle ??

Well-defined Problems and Solutions

Path Cost Function

- A function that assigns a cost to a path (sequence of actions).
 - Is often denoted by g . Usually, it is the sum of the costs of the individual actions along the path (from one state to another state)
 - Measure path cost of a sequence of actions in the state space. For example, we may prefer paths with fewer or less costly actions
- **Example:**
 - Route finding problem: Path cost from initial to goal state
 - Coloring problem: One for each transition from state to state till goal state is reached
 - 8-puzzle?

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- **States:** locations of tiles
- **Actions:** move blank left, right, up, down
- **Goal Test:** goal state (given)
- **Path Cost:** 1 per move

Searching for Solution

- A **solution** to a problem is an action sequence that leads from the initial state to a goal state.
- **Solution quality** is measured by the **path cost** function, and an **optimal solution** has the lowest path cost among all solutions.
- An agent with several immediate options of unknown value can decide what to do by first examining different possible sequences of actions that lead to states of known value, and then choosing the best sequence.
 - This process of looking for such a sequence is called search
 - This is done by a search through the state space.
- Examine different possible sequences of actions & states, and come up with specific sequence of operators/actions that w take you from the initial state to the goal state

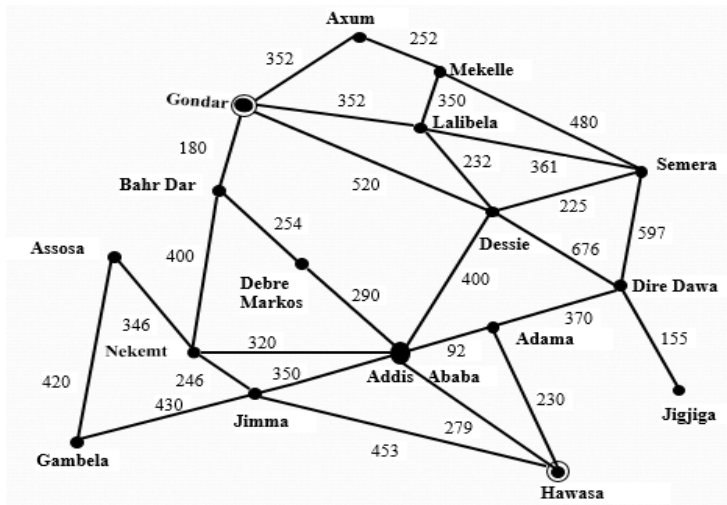
Searching in State Space

- choose min-cost/max-profit node/state in the state space,
- test the state to know whether we reached to the goal state or not,
- if not, expand the node further to identify its successor.

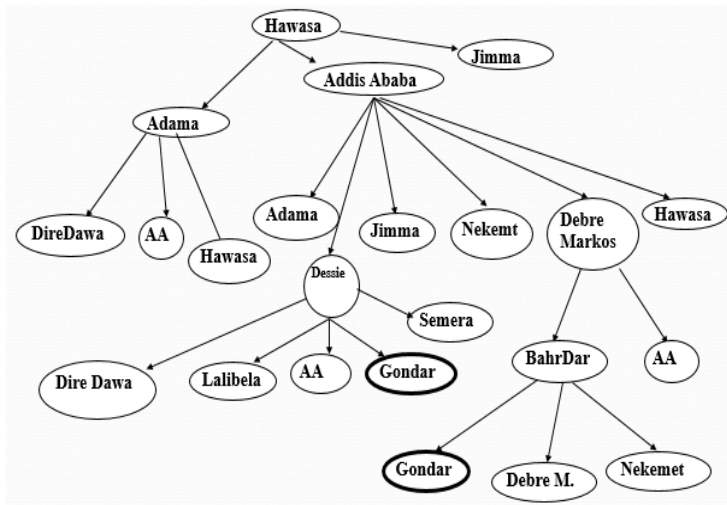
Search Tree

- The searching process is like building the search tree that is super imposed over the state space
- A search tree is a representation in which nodes denote paths and branches connect paths. The node with no parent is the root node. The nodes with no children are called leaf nodes.
- Given state S and valid actions being at S , the set of next state generated by executing each action is called successor of S
- The Successor-Function generate all the successors states and the action that leads moves the current state into the successor state

Example: Road Map of Ethiopia



Tree search example: Route finding Problem



Outline

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

Search Algorithms

- Problem solving in AI may be characterized as a systematic search through a range of possible actions in order to reach some predefined goal or solution.
- Search algorithms are one of the most important areas of Artificial Intelligence.
- A search algorithm takes a problem as input and returns the solution in the form of an action sequence.
 - Once the solution is found, the actions it recommends can be carried out.
 - This phase is called as the execution phase.
 - After formulating a goal and problem to solve the agent cells a search procedure to solve it.

Search Algorithm Terminologies

Functions needed for conducting search

- **Generator (or successors) function:** Given a state and action, produces its successor states (in a state space)
- **Tester (or IsGoal) function:** Tells whether given state **S** is a goal state **IsGoal(S) → True/False**
 - IsGoal and Successors functions depend on problem domain.
- **Merge function:** Given successor nodes, it either append, prepend or arrange based on evaluation cost
- **Path cost:** function assigning a numeric cost to each path; either from initial node to current node and/or from current node to goal node

Search Algorithm Terminologies

Two lists maintained during searching

- **OPEN list:** stores the nodes we have seen but not explored
- **CLOSED list:** the nodes we have seen and explored
- The set of all child nodes available for expansion at any given point is called the **frontier (open list or fringe)**
 - The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.
 - The way to avoid exploring redundant paths is to remember where one has been.
 - To do this a data structure called the **explored set** (also known as the **closed list**), which remembers every expanded node, can be used
- Generally, search proceeds by examining each node on the **OPEN list**, performing some expansion operation that adds its children to the **OPEN list**, and moving the node to the **CLOSED list**.

Search Algorithm Basic Structure

Algorithm 1 Pseudocode for Search Algorithm

- 1: **function** GeneralSearch(**problem**) **returns** a solution or failure
 - 2: *initialize* the **frontier** using the initial state of problem
 initialize the **explored set** to be *empty*
 - 3: **loop do**
 - 4: **If** the frontier is empty **then return failure**
 - 5: choose a leaf node and remove it from the frontier
 - 6: **if** the node contains a goal state **then return** the corresponding solution
 - 7: *add the node to the explored set*
 - 8: expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set
-

- Search algorithms all share this basic structure; they vary primarily according to how they choose which state to expand next—the so-called **search strategy**.

Example Search Algorithm Implementation

Algorithm 2 Example for a Search Algorithm Implementation

- 1: **Input:** a given problem (Initial State + Goal state + transit states and operators)
- 2: **Output:** returns optimal sequence of actions to reach the goal.
- 3: **function** GeneralSearch (problem, strategy)
- 4: open = (initialState); //put initial state in the List
- 5: closed = {}; //maintain list of nodes examined earlier
- 6: **while** (not (empty (open)))
- 7: f = remove_first(open);
- 8: **if** IsGoal (f) then return (f);
- 9: closed = append (closed, f);
- 10: succ = Successors (f);
- 11: l = not-in-closed (succ, closed);
- 12: open = merge (rest(open), l); //append or prepend l to open list
- 13: end while
- 14: return ('fail')
- 15: **end** GeneralSearch

Algorithm Evaluation

- The output of a problem-solving algorithm is either failure or a solution.
 - Some algorithms might get stuck in an infinite loop and never return an output.
- Search Algorithms are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution (the depth of the shallowest goal node)
 - m : maximum depth of the state space

Outline

① Problem Solving Agents

② Search Algorithm

③ Search Strategies

Uninformed Search

Informed Search

Search Strategies

- Search strategy gives the order in which the search space is examined
- Generally, searching strategies can be classified in to two as **uninformed** and **informed** search strategies

Uninformed search (blind search) strategies

- Uninformed search strategies use only the information available in the problem definition
- They do not need domain knowledge that guide them to the right direction towards the goal
- They have no information about the number of steps or the path cost from the current state to the goal
- They can distinguish the goal state from other states
- They are still important because there are problems with no additional information.

Search Strategies

Informed (heuristic) search

- Informed search is a strategy that uses information about the cost that may incur to achieve the goal state from the current state.
- The information may not be accurate. But it will help the agent to make better decision
 - This information is called **heuristic information**
- Have problem-specific knowledge (knowledge that is true from experience)
- Have knowledge about how far are the various state from the goal
- Can find solutions more efficiently than uninformed search

Uninformed Search

- There are different kinds of such search strategies:
 - Breadth-first search
 - Uniform-cost search
 - Depth-first search
 - Depth-limited search
 - Iterative deepening search
 - etc.
- All search strategies are distinguished by the order in which nodes are expanded.

Breadth-first Search (BFS)

- Breadth-first search is a simple strategy in which the root node is expanded first, then all the successors of the root node are expanded next, *then their successors, and so on*.
- In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.
- Breadth-first search is an instance of the general search algorithm in which the *shallowest unexpanded node is chosen for expansion*.
 - This is achieved very simply by using a **FIFO queue** for the **frontier**.
- Thus, new nodes (which are always deeper than their parents) go to the back of the queue, and old nodes, which are shallower than the new nodes, get expanded first.

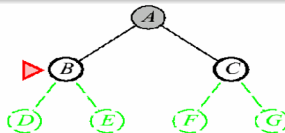
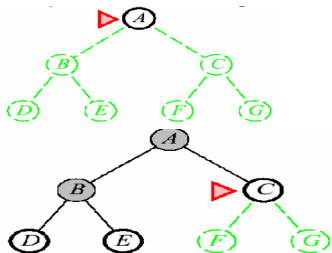
Breadth-first Search (BFS)

- Expand shallowest unexpanded node, i.e. expand all nodes on a given level of the search tree before moving to the next level

BFS Implementation

Use queue data structure to store the list:

- Fringe (open list)* is a FIFO queue, i.e., new successors go at the end
- Expansion: put successors at the end of queue
- Pop nodes from the front of the queue



Properties of BFS

- **Complete? Yes** (if b is finite, which is true in most cases)
 - it is complete if the shallowest goal node is at some finite depth d , breadth-first search will eventually find it after expanding all shallower nodes (provided the branching factor b is finite)
- **Optimal? Yes** (if cost = constant (k) per step)
 - breadth-first search is optimal if the path cost is a non-decreasing function of the depth of the node. For example, when all actions have the same cost.
- **Time?** $1 + b + b^2 + b^3 + \dots + b^d = O(b^{d+1})$
 - at depth value = i , there are b^i nodes expanded for $i \leq d$
 - If the algorithm were to apply the goal test to nodes when selected for expansion, rather than when generated, the whole layer of nodes at depth d would be expanded before the goal was detected and the time complexity would be $O(b^{d+1})$
 - If the algorithm were to apply the goal test to nodes when generated, rather than when selected for expansion, the whole layer of node depth d would not be expanded and the time complexity would be $O(b^d)$

Properties of BFS

Space? $O(b^d)$ (keeps every node in memory)

- Every node that is generated must remain in memory, because it is either part of the fringe or is an ancestor of a fringe node. The space complexity is, therefore, the same as the time complexity i.e., $O(b^{d+1})$ or $O(b^d)$ based on the goal test function placed in the breadth-first search algorithm
- This is a major problem for real problem

Algorithm for Breadth first search

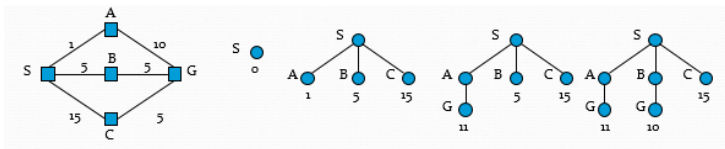
Algorithm 3 BFS

```

1: function BFS (problem) {
2:   open = (C_0); //put initial state C_0 in the List
3:   closed = {}; //maintain list of nodes examined earlier
4:   while (not (empty (open))) {
5:     f = remove_first(open);
6:     if IsGoal (f) then return (f);
7:     closed = append (closed, f);
8:     l = not-in-closed (Successors (f), closed );
9:     open = merge (rest(open), l); //append l to the open list
10:  }
11:  return ('fail')
12: }
```

Uniform Cost Search (UCS)

- Uniform-cost search expands the node n with the lowest path cost $g(n)$. This is done by storing the frontier as a priority queue ordered by total path cost.
- Nodes in the open list keep track of total path length from start to that node
- Open list kept in priority queue ordered by path cost
- The goal of this technique is to find the shortest path to the goal in terms of cost.
 - It modifies the BFS by always expanding least-cost unexpanded node



Uniform Cost Search (UCS)

Implementation

- Nodes in list keep track of total path length from start to that node
- List kept in priority queue ordered by path cost
- Uniform-cost search does not care about the number of steps a path has, but only about their total cost.

Uniform Cost Search (UCS)

Properties

It finds the cheapest solution if the cost of a path never decrease as we go along the path i.e. $g(\text{sucessor}(n)) \geq g(n)$ for every node n .

- **Completeness** is guaranteed provided the cost of every step exceeds some small positive constant ϵ . (*if all cost of actions > 0*)
- It is **optimal** if all cost of actions > 0 .
 - Nodes expanded in increasing order of $g(n)$
- Uniform-cost search is guided by path costs rather than depths, so its complexity is not easily characterized in terms of b and d .
- Instead, let C^* be the cost of the optimal solution, and ass that every action costs at least ϵ .

Uniform Cost Search (UCS)

Properties

- **Time?** # of nodes with $g \leq$ cost of optimal solution
 - Now you can ask, In the worst case, what will be the number of nodes that exist all of which has step cost $= \epsilon$, branching factor b and path cost $\leq C^*$. The resulting tree will have depth value (C^*/ϵ)
 - Hence the total node will be $(b^{C^*/\epsilon})$. Therefore, time complexity becomes $O(b^{(C^*/\epsilon)})$
- **Space?** # of nodes with $g \leq$ cost of optimal solution, $O(b^{(C^*/\epsilon)})$
- When all step costs are equal, $b^{1+(C^*/\epsilon)}$ is just b^{d+1} .

Algorithm for Uniform Cost Search

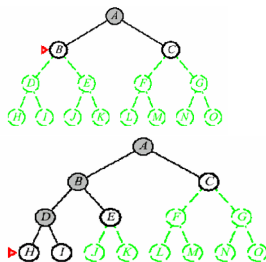
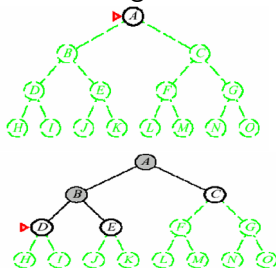
Algorithm 4 UCS

```

1: function UCS (problem) {
2:   open = (C_0); //put initial state C_0 in the List
3:   g(s)=0;
4:   closed = {}; //maintain list of nodes examined earlier
5:   while (not (empty (open))) {
6:     f = remove_first(open);
7:     if IsGoal (f) then return (f);
8:     closed = append (closed, f);
9:     l = not-in-closed (Successors (f), closed );
10:     $g(f, l_i) = g(f) + c(f, l_i)$ 
11:    open = merge(rest(open), l, g(f, l_i)); //keep the open list sorted in
                                           ascending order by edge cost
12:  }
13:  return ('fail')
14: }
```

Depth-first Search (DFS)

- It always expands the deepest node from the current frontier (open list) of the search tree
- It uses a **LIFO** queue for the open list (frontier)
 - the most recently generated node is chosen for expansion, which is the deepest unexpanded node (it is deeper than its parent)
- Expand one of the node at the deepest level of the tree.
 - Only when the search hits a non-goal dead end does the search go back and expand nodes at shallower levels



Depth-first Search (DFS)

Implementation: treat the list as stack

- It uses a LIFO for the open list (frontier)
- Expansion: push successors at the top of stack
- Pop nodes from the top of the stack

Properties of Depth-first Search (DFS)

- **Complete?** **No**: fails in infinite-depth spaces, spaces with loops
 - Depth-first search is complete if the state space is finite
 - If the state space is infinite then it is not complete
- **Optimal?** **No**
- **Time?** $O(b^m)$, where **b** is branching factor and **m** is maximum depth
 - It may generate all the nodes in the search tree, terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- **Space?** $O(bm)$, i.e., linear space!
 - DFS needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes for each node on the path.
 - Once a node has been expanded, it can be removed from memory as soon as all its descendants have been fully expl

Algorithm for Depth-first Search (DFS)

Algorithm 5 DFS

```

1: function DFS (problem) {
2:   open = (C_0); //put initial state C_0 in the List
3:   closed = {}; //maintain list of nodes examined earlier
4:   while (not (empty (open))) {
5:     f = remove_first(open);
6:     if IsGoal (f) then return (f);
7:     closed = append (closed, f);
8:     l = not-in-set (Child-Nodes(f), closed );
9:     open = merge ( rest(open), l); //prepend to the list
10:  }
11:  return ('fail')
12: }
```

Depth-limited Search

- We can see that the breadth search is complete which can be taken as its advantage though its space complexity is the worst
- Similarly the depth first search strategy is best in terms of space complexity even if it is the worst in terms of its completeness and time complexity compared to breadth first search
- Hence, we can find an algorithm that incorporate both benefits and avoid the limitation
- Such algorithm is called depth limited search and its improved version is called iterative deepening search strategy.
- These two strategies will be explored in the following sections

Depth-limited Search

Definition

Depth-first search with depth limit l , will truncate all nodes having depth value greater than l from the search space and apply depth first search on the rest of the structure

- It return **solution** if solution exist, if there is no solution
- It return **cutoff** if $l < m$, **failure** otherwise

Properties

Complete? **No** (fail if all solution exist at $depth > l$)

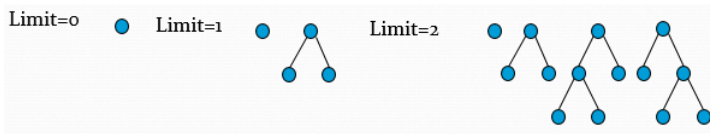
Time? $O(b^l)$

Space? $O(bl)$

Optimal? **No**

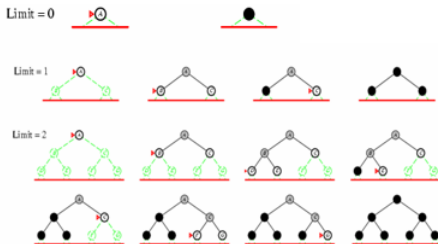
Iterative Deepening Search (IDS)

- IDS solves the issue of choosing the best depth limit by trying all possible depth limit:
 - Perform depth-first search to a bounded depth d , starting at $d = 1$ and increasing it by 1 at each iteration.
- This search combines the benefits of DFS and BFS
 - DFS is efficient in space, but has no path-length guarantee
 - BFS finds min-step path towards the goal, but requires memory space
 - IDS performs a sequence of DFS searches with increasing depth-cutoff until goal is found



Iterative Deepening Search (IDS)

- It is the search strategy that combines the benefits of depth-first and breadth-first search
- Like depth-first search, its memory requirement is $O(bd)$ to be precise.
- Like breadth-first search, it is complete when the branching factor is finite and optimal when the path cost is a non-decreasing function of the depth of the node.



Algorithm for IDS

Algorithm 6 IDS

```

1: function IDS (problem) {
2:   open = (C_0); //put initial state C_0 in the List
3:   closed = {}; //maintain list of nodes examined earlier
4:   while (not reached maxDepth) {
5:     while (not (empty (open))) {
6:       f = remove_first(open);
7:       if IsGoal (f) then return (f);
8:       closed = append (closed, f);
9:       l = not-in-set (Child-Nodes(f), closed );
10:      if (depth(l) < maxDepth) then
11:        open = merge ( rest(open), l); //prepend to the list
12:      }
13:    }
14:    return ('fail')
15:  }
```

Comparing Uninformed Search

Strategies	Complete	Optimal	Time complexity	Space complexity
Breadth first search	yes	yes	$O(b^d)$	$O(b^d)$
Depth first search	no	no	$O(b^m)$	$O(bm)$
Uniform cost search	yes	yes	$O(b^d)$	$O(b^d)$
Depth limited search	if $l \geq d$	no	$O(b^l)$	$O(bl)$
Iterative deepening search	yes	yes	$O(b^d)$	$O(bd)$

- b is branching factor,
- d is depth of the shallowest solution,
- m is the maximum depth of the search tree,
- l is the depth limit

Summary

- Before an agent can start searching for solutions, it must **formulate a goal** and then use that goal to **formulate a problem**.
- A problem consists of five parts: The **state space**, **initial situation**, **actions**, **goal test**, and **path costs**.
- A path from an initial state to a goal state is a **solution**.
- A general search algorithm can be used to solve any problem.
 - Specific variants of the algorithm can use different search strategies.
- Search algorithms are judged on the basis of **completeness**, **optimality**, **time complexity**, and **space complexity**.
- Iterative deepening in general is the preferred uninformed search method when there is a large search space and the depth of solution is not known.

Informed Search

Lesson Objectives

- Examine the main characteristics of informed search strategies
- Describe how informed search strategies applied for solving problems
- Examine the efficiency of informed search strategies
- Compare and contrast greedy and A* search strategies

What is an Informed Search?

- Search efficiency would improve greatly if there is a way to order the choices so that the most promising are explored first.
 - This requires domain knowledge of the problem (i.e. heuristic) to undertake focused search
- Informed search is a strategy that uses information about the cost that may incur to achieve the goal state from the current state.
- The information may not be accurate. But it will help the agent to make better decision
 - This information is called heuristic information
- The generic name to the class of informed search is called Best First Search

Heuristic Function

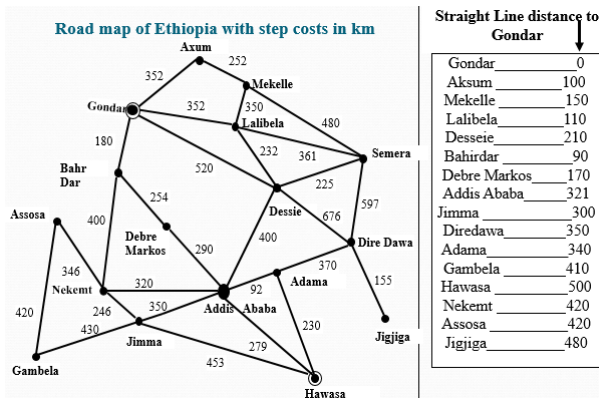
Definition

- Most best-first algorithms include as a component of f a heuristic function, denoted $h(n)$:
 - **$h(n)$** = estimated cost of the cheapest path from the state at node n to a goal state
- Heuristic functions are the most common form in which additional knowledge of the problem is imparted to the search algorithm.
- Heuristic functions estimates the **goodness** of a node n , based on domain specific information that is computable from the current state description.

Heuristic Function

Heuristic function for Route finding

- Route finding: $h(n)$: straight line distance from the goal state



Heuristic Function

Heuristic function for 8-puzzle

- ① **$h1(n)$** : Number of mismatched tiles and
- ② **$h2(n)$** : Manhattan or city block distance (sum of distances each tile is from its goal position)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

— $h1(S) = 6$

— $h2(S) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = 14$

Best First Search

- It is a generic name to the class of informed methods
- The two best first approaches to find the shortest path:
 - **Greedy search:** minimizes estimated cost to reach a goal
 - **A*-search:** minimizes the total path cost
- When expanding a node **n** in the search tree, greedy search uses the estimated cost to get from the current state to the goal state, define as **$h(n)$** .
 - In route finding problem
- We also possess the sum of the cost to reach that node from the start state, define as **$g(n)$** .
 - In route finding problem this is the sum of the step costs for the search path.
- For each node in the search tree, an evaluation function $f(n)$ can be defined as the sum of these functions.

$$f(n) = g(n) + h(n)$$

Admissibility

- Search algorithms (such as greedy and A^* - search) are admissible when the heuristic function never over estimated the actual cost so that
 - the algorithm always terminates in an optimal path from the initial state to goal node if one exists.
- Check whether estimated cost $h(n)$ is not overestimated
 - $g(n)$: Actual cost of shortest path from n to z
 - $h(n)$ is said to be an admissible heuristic function if for all n , $h(n) \leq g(n)$
 - The closer estimated cost to actual cost the fewer extra nodes that will be expanded
 - Using an admissible heuristics guarantees that the solution found by searching algorithm is optimal

Greedy Search

- Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.
- It evaluates nodes by using just the heuristic function; that is, $f(n) = h(n)$, i.e., the evaluation function is equal to heuristic
- The greedy search strategies doesn't take the costs so far into account, it uses a heuristic function $h(n)$ alone to guide the search
 - $h(n) = 0$ if node n is the goal state
 - Otherwise $h(n) \geq 0$; an estimated cost of the cheapest path from the state at node n to a goal state

Greedy Search

Properties

- **Complete?** **Yes** if repetition is controlled otherwise it can get stuck in loops
- **Time?** $O(b^m)$, but a good heuristic can give dramatic improvement
- **Space?** $O(b^m)$, keeps all nodes in memory
- **Optimal?** **No**

A*-search Algorithm

- The **problem** with greedy search is that it didn't take costs so far into account.
 - We need a search algorithm (like A^* search) that takes into consideration this cost so that it avoids expanding paths that are already expensive
- It considers both estimated cost of getting from n to the goal node, $h(n)$, and cost of getting from initial node to node n , $g(n)$
- Apply three functions over every nodes
 - $g(n)$: Cost of path found so far from initial state to n
 - $h(n)$: Estimate cost of shortest path from n to goal state
 - $f(n)$: estimated cost of the cheapest solution through n
 - Evaluation function $f(n) = h(n) + g(n)$

A*-search Algorithm

Implementation

- Expand the node for which the evaluation function $f(n)$ is lowest
- Rank nodes by $f(n)$ that goes from the start node to goal node via given node
- The algorithm is identical to **UNIFORM-COST-SEARCH** except that A^* uses $h(n) + g(n)$ instead of $g(n)$ alone

A*-search Algorithm

Properties

A* search is complete, optimal, and optimally efficient for any given admissible heuristic function

- **Complete?** **Yes**-It guarantees to reach to goal state
- **Time?** and **Space?**
 - It keeps all generated nodes in memory
 - Both time and space complexity is exponential
 - Time and space complexity of heuristic algorithms depends on the quality of the heuristic function.
 - Worst case is exponential
- **Optimal?** If the given heuristic function is admissible, then A* search is optimal.